

Introduction to the Laboratory

ARM Kit with MCXC143VFT

Juraj Sýkora

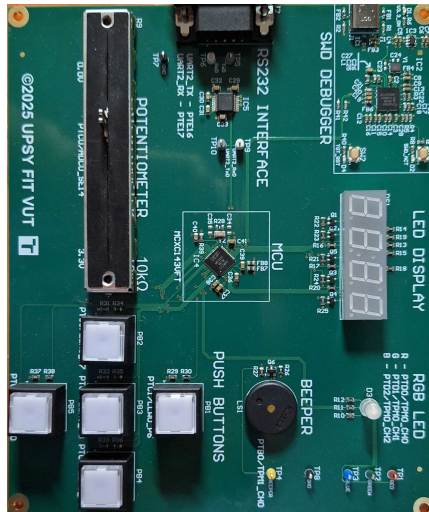
Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2, 612 00 Brno – Královo Pole
xsykorj00@vutbr.cz



May 13, 2026

- 32-bit ARM MCU laboratory kit
- Schematic: debug interface and power supply
- Schematic: target MCU MCXC143VFT and peripherals
- MCUXpresso IDE – project creation
- MCUXpresso IDE – build, flash and debug

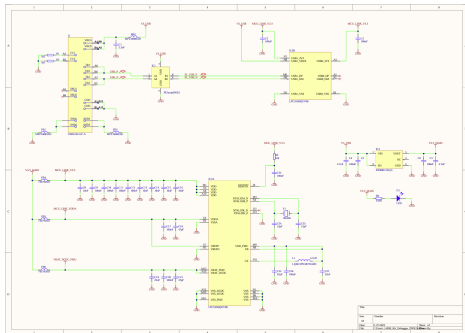
- Kits are located in laboratory **L306**
- Target MCU: **NXP MCXC143VFT**
 - ARM **Cortex-M0+** core, up to **48 MHz**
 - **256 kB** Flash, **32 kB** SRAM
 - 48-pin LQFP package
- On-board debugger: **MCU-Link v3** (NXP LPC55S69JEV98)
- **USB-C** connector
 - power + CMSIS-DAP/SWD debug
- 3.3 V main supply via **RT9080-33** LDO



(Kit in the labs)

Key components:

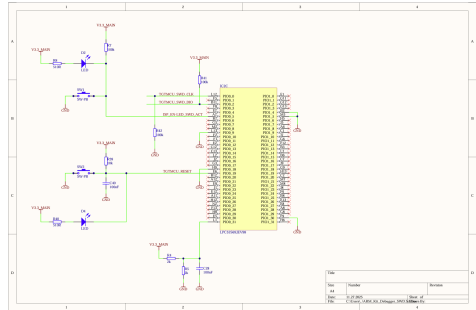
- **USB4105-GF-A** – USB-C receptacle
- **RCIamp0502A** – USB ESD protection
- **LPC55S69JEV98** – MCU-Link v3 chip
- **RT9080-33GJ5** – 3.3 V LDO regulator
- **MPZ1608S331** – ferrite bead EMI filters
- **Y1** – 16 MHz crystal for MCU-Link



*(Schematic:
ARM_Kit_Debugger_PWR)*

SWD connections (LPC55S69 → MCXC143):

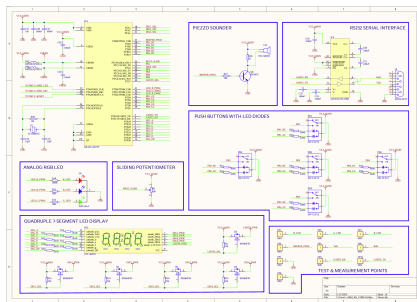
- **TGTMCU_SWD_CLK / _DIO**
– SWD clock and data lines
- **TGTMCU_RESET** – target reset
- **ISP_EN / LED_SWD_ACT**
– status
- **SW1 / SW2** – user buttons (debugger side)
- **R4, R5** (2 kΩ) – SWD pull-ups
- **D2 / D4** – activity LEDs



(Schematic:
ARM_Kit_Debugger_SWD)

Key pin assignments:

- **PTA0/SWD_CLK**,
PTA3/SWD_DIO,
PTA20/RESET_b – SWD debug interface
- **PTE16/UART2_TX**,
PTE17/UART2_RX
– RS-232 via MAX3221E
- **PTD0-2 / TPM0_CH0-2** – RGB LED PWM
- **PTB0 / TPM1_CH0** – buzzer PWM
- **PTC0 / ADC0_SE14** – sliding potentiometer
- **VDDA / VREFH / VREFL**
– analog supply and reference



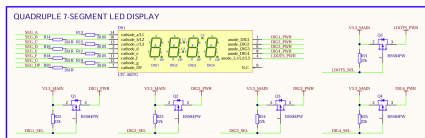
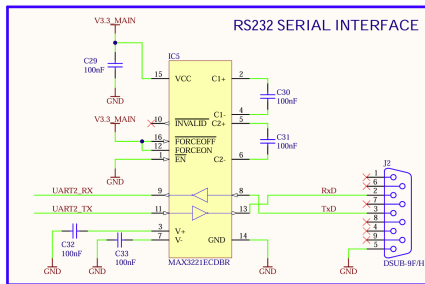
(Schematic: *ARM_Kit_CORE – IC4*)

RS-232 serial interface:

- Transceiver
MAX3221ECDBR
- Connector **DSUB-9F/H**
- UART2_TX/RX $\leftrightarrow$ TxD/RxD

4-digit 7-segment display (DS1 – LTC-4627G):

- Common-anode; digit drive:
Q1–Q4 (BSS84PW P-FETs),
Q5 drives colon dots (LDOTS)
- Segment signals:
SEG_A–G, SEG_DP
- Current-limiting resistors R13–R20 (261 Ω)



(Schematic: RS-232 + 7-SEG)

Schematic – Buttons, RGB LED, Buzzer, Potentiometer

Push buttons PB1–PB5 (SW-ULP12):

- **PBx_RC / PBx_GC** – GPIO and LED anode

Analog RGB LED (D3 – WP154A4):

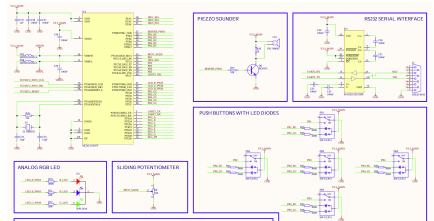
- **LED_R/G/B_PWM** driven by **TPM0_CH0–2**
- $R10=300\ \Omega$, $R11 / R12=20\ \Omega$

Piezo buzzer (LS1):

- NPN driver **Q6** (BC847C)
- Signal: **BEEPER_PWM** on **PTB0**

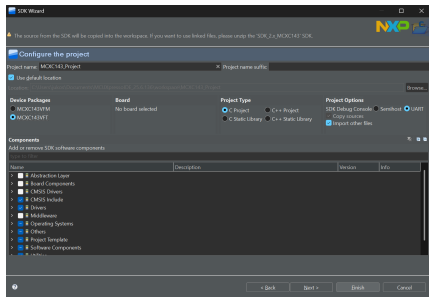
Sliding potentiometer:

- **RPOT_SLIDE** → **ADC0_SE14**



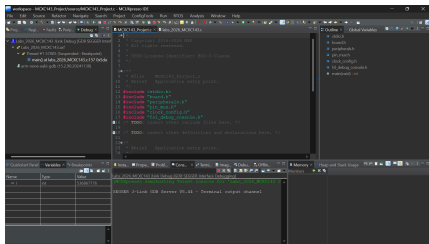
(Schematic: *BUTTONS + RGB + BUZZER + POT*)

- 1 **File** → **New** → Create a new C/C++ project
- 2 Select **NXP MCUXpresso SDK Project**
- 3 Choose **SDK** for **MCXC143VFT**
(download from `mcuxpresso.nxp.com` if not yet installed)
- 4 Enter a **project name**; set the workspace,
e.g. `D : / IMP_Labs /` or
`P : / IMP_Labs /`
- 5 Select language **C**; leave the default linker script
- 6 Click **Finish**



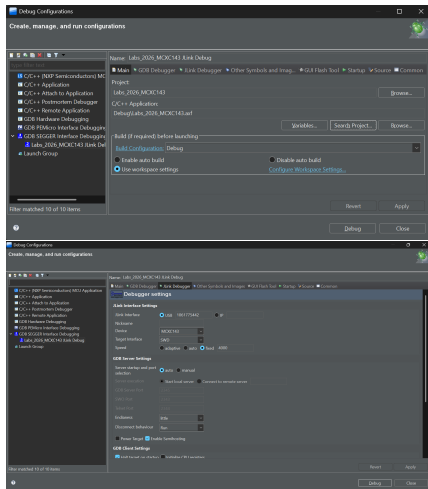
(Screenshot: New Project Wizard)

- **Build** (*hammer icon*)
 - compiles sources and links the binary
- **Debug** (*bug icon*)
 - starts an interactive debug session
- **Perspective switcher**
 - toggle between C/C++ (edit) and Debug views
- **Console** panel
 - build output; check here for errors
- **Variables / Registers / Memory**
 - inspect MCU state at runtime



(Screenshot: MCUXpresso main window)

- 1 Arrow next to **Debug** → Debug Configurations. . .
- 2 Expand **GDB SEGGER Interface Debugging**; select or create the project entry
- 3 **Main** tab – verify:
 - Project – your project name
 - C/C++ Application path
- 4 **JLink Debugger** tab – verify:
 - Device: MCXC143
 - Target Interface: SWD
 - JLink Interface: USB
- 5 Click **Debug**
- 6 Resume execution: **F8** or green **Resume** button



(Screenshot: Debug Configurations)

- **Resume (F8)** – starts or continues execution on the MCU
- **Suspend** – pauses execution; inspect variables, registers, memory
- **Terminate (Ctrl+F2)** – ends the debug session
Confirm the entry in the **Debug** view turns **grey**
A **red** entry means the session is still active – cannot be restarted!
- After terminating, click **C/C++** perspective to return to the editor

Stuck debug session

If the debug thread hangs, right-click `<terminated>` in the **Debug** view and choose **Terminate and Remove**.

```
#include "MCXC143.h"

static void delay(volatile uint32_t n) {
    while (n--);
}

int main(void) {
    /* Disable watchdog explicitly */
    SIM->COPC = SIM_COPC_COPT(0);

    /* Enable clock gate for Port B */
    SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;

    /* PTB0 -> GPIO output (MUX = 1) */
    PORTB->PCR[0] = PORT_PCR_MUX(1);
    GPIOB->PDDR |= (1u << 0);
    GPIOB->PCOR = (1u << 0); /* start LOW */

    for (;;) {
        GPIOB->PSOR = (1u << 0); /* PTB0 high */
        delay(50000u);
        GPIOB->PCOR = (1u << 0); /* PTB0 low */
        delay(50000u);
    }
    return 0;
}
```

Notes:

- Header `MCXC143.h` selected for `MCXC143`
- Use atomic `PSOR/PCOR` instead of read-modify-write on `PDOR`
- Watchdog disabled explicitly via `SIM->COPC`
- **BEEPER_PWM** is wired to **PTB0** on the kit (see schematic `ARM_Kit_CORE`)

```

/* Override in your application code */
void DMA0_IRQHandler(void);
void DMA1_IRQHandler(void);
void DMA2_IRQHandler(void);
void DMA3_IRQHandler(void);
void FTFE_IRQHandler(void);          /* Flash done */
void LVD_LVW_IRQHandler(void);
void LLWU_IRQHandler(void);
void I2C0_IRQHandler(void);
void SPI0_IRQHandler(void);
void UART2_IRQHandler(void);        /* RS-232 */
void ADC0_IRQHandler(void);
void CMP0_IRQHandler(void);
void TPM0_IRQHandler(void);         /* RGB PWM */
void TPM1_IRQHandler(void);         /* Buzzer PWM */
void RTC_IRQHandler(void);
void PIT_IRQHandler(void);
void LPTMR0_IRQHandler(void);
void PORTA_IRQHandler(void);
void PORTB_IRQHandler(void);
void PORTC_PORTD_IRQHandler(void); /* C and D */
void PORTE_IRQHandler(void);

```

- Names defined in **startup_mcxc143.c**
- Override: define the same function in your code (weak symbol replacement)
- PORTC and PORTD share one IRQ vector**
 $\Rightarrow$ `PORTC_PORTD_IRQHandler`
- Cortex-M0+ NVIC: **4 priority levels** (2-bit priority field)
- CMSIS API:
`NVIC_EnableIRQ()`
`NVIC_SetPriority()`

Thank You For Your Attention!